# NestJs

Mitsiu Alejandro Carreño Sarabia

## Course structure Partial test

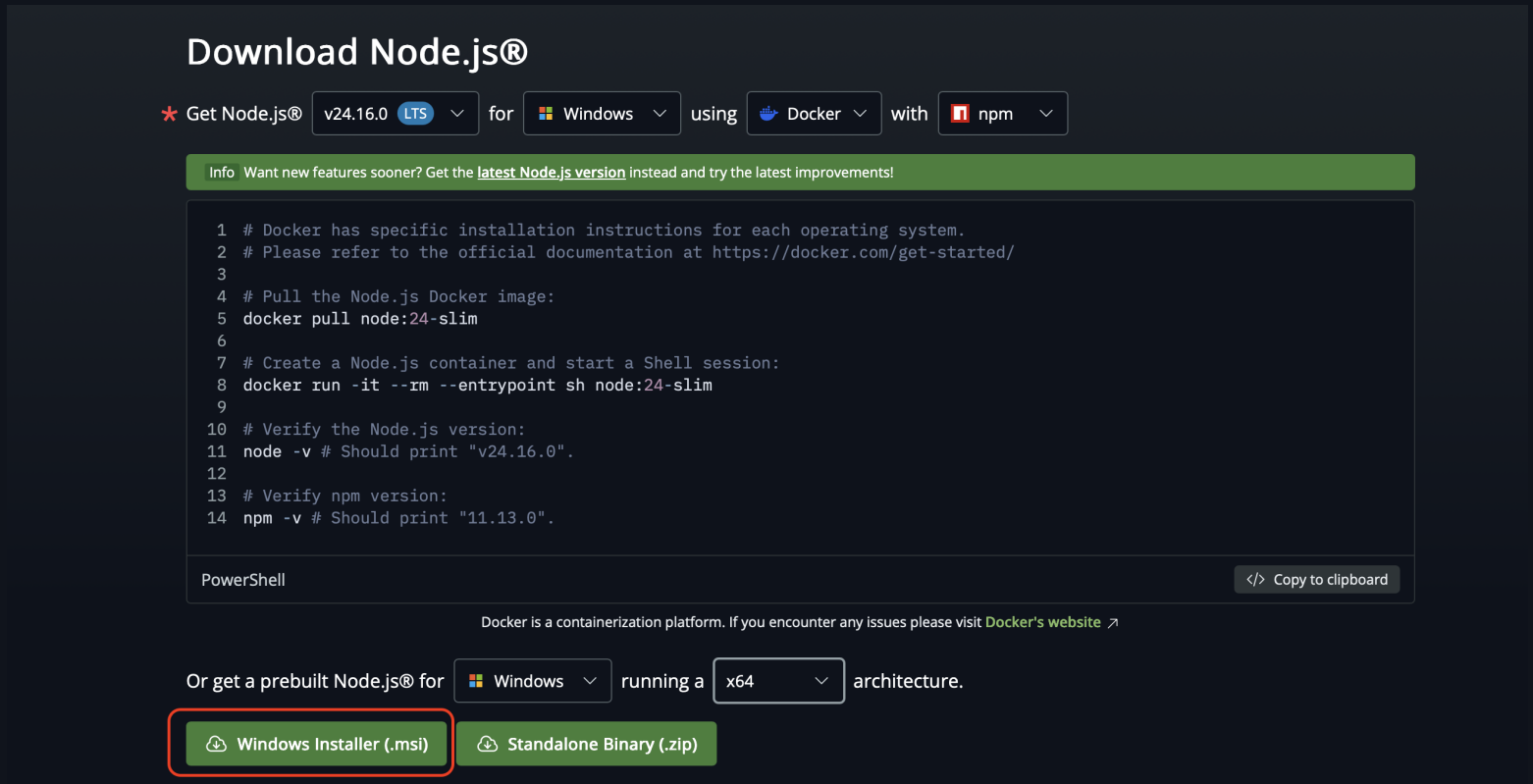
- 2nd Partial = 30%
  - Autoevaluation = 5%
  - Theorical evaluation = 40%
  - Practical evaluation = 55%

# Agenda

- Requirements
- Installation
- Code execution
- Code exploration

# Requirements

Install node (https://nodejs.org/en/download)




* You can use docker if you know how to use it.

## Requirements

Confirm installation with the commands:

```
node --version
npm --version
```

## Installation

We are going to use nestJs framework to develop our API's

```
npm install -g @nestjs/cli
nest new Ex3-nestjs-intro

# Alternatively
npx @nestjs/cli new Ex3-nestjs-intro
```

You can use either npm or yarn (if installed)

```
? Which package manager would you ❤  to use?
❯ npm
  yarn
  pnpm
```

## ▒▒▒▒▒▒ Code execution

How do we run our application?

```
✔ Installation in progress... ☕

🚀  Successfully created project ex3-nestjs-intro
👉  Get started with the following commands:

$ cd ex3-nestjs-intro
$ yarn run start


                Thanks for installing Nest 🙏
Please consider donating to our open collective
       to help us maintain this package.


🍷  Donate: https://opencollective.com/nest
```

## Code execution

How do we run our application?

```
✔ Installation in progress... ☕

🚀  Successfully created project ex3-nestjs-intro
👉  Get started with the following commands:

$ cd ex3-nestjs-intro
$ npm run start


                Thanks for installing Nest 🙏
Please consider donating to our open collective
       to help us maintain this package.


🍷  Donate: https://opencollective.com/nest
```

## ▒▒▒▒▒ Code execution

We can see the following logs:

```
$ nest start
[Nest] 33992  - LOG [NestFactory] Starting Nest
application...
[Nest] 33992  - LOG [InstanceLoader] AppModule
dependencies initialized +3ms
[Nest] 33992  - LOG [RoutesResolver]
AppController {/}: +1ms
[Nest] 33992  - LOG [RouterExplorer] Mapped {/,
GET} route +1ms
[Nest] 33992  - LOG [NestApplication] Nest
application successfully started +0ms
```

Based on them how do we access our api from the browser and postman/bruno?

## ▒▒▒▒▒ Code execution

Little pro tip:

```
npm run start:dev
```

;)

## ▒▒▒▒▒▒▒ Code exploration

Let's dig deeper into what did this command do?

```
nest new Ex3-nestjs-intro
```

- What's the new folder name?

```
ex3-nestjs-intro
├── dist
├── eslint.config.mjs
├── nest-cli.json
├── node_modules
├── package.json
├── README.md
├── src
├── test
├── tsconfig.build.json
├── tsconfig.json
└── yarn.lock
```

## ▒▒▒▒▒▒▒ Code exploration

Node modules and a few other boilerplate files will be installed, and a src/ directory will be created and populated with several core files.

```
src
├── app.controller.spec.ts
├── app.controller.ts
├── app.module.ts
├── app.service.ts
└── main.ts
```

## ▒▒▒▒▒▒▒ Code exploration

| File | Description |
| --- | --- |
| app.controller.ts | A basic controller with a single route. |
| app.controller.spec.ts | The unit tests for the controller. |
| app.module.ts | The root module of the application. |
| app.service.ts | A basic service with a single method. |
| main.ts | The entryfile create a Nest app instance |

That's a lot of buzzwords let's start with the part we are already familiar...

## ▒▒▒▒▒▒▒ Code exploration

Let's begin with the file `src/app.controller.ts`:

> A basic controller with a single route.

```
import { Controller, Get } from '@nestjs/common';
import { AppService } from './app.service';

@Controller()
export class AppController {
  constructor(private readonly appService: AppService) {}

  @Get()
  getHello(): string {
    return this.appService.getHello();
  }
}
```

## ▒▒▒▒▒▒▒ Code exploration

```
1 import { Controller, Get } from '
  @nestjs/common';
2 import { AppService } from './app.service';
3
4 @Controller()
5 export class AppController {
6   constructor(private readonly appService:
  AppService) {}
7
8   @Get()
9   getHello(): string {
10     return this.appService.getHello();
11   }
12 }
```

export allow to use the class in other files

## ▒▒▒▒▒▒▒ Code exploration

```
1 import { Controller, Get } from '
  @nestjs/common';
2 import { AppService } from './app.service';
3
4 @Controller()
5 export class AppController {
6   constructor(private readonly appService:
  AppService) {}
7
8   @Get()
9   getHello(): string {
10     return this.appService.getHello();
11   }
12 }
```

`<function>(): <type>` express the function return data type

## ▒▒▒▒▒▒▒ Code exploration

```
1 import { Controller, Get } from '
  @nestjs/common';
2 import { AppService } from './app.service';
3
4 @Controller()
5 export class AppController {
6   constructor(private readonly appService:
  AppService) {}
7
8   @Get()
9   getHello(): string {
10     return this.appService.getHello();
11   }
12 }
```

private only within the class can be accessed
readonly prevents modification of that property after initialization
<variable>:<type> in this context (not JSON) a variable has type

```
1 import { Controller, Get } from '
  @nestjs/common';
2 import { AppService } from './app.service';
3
4 @Controller()
5 export class AppController {
6   constructor(private readonly appService:
  AppService) {}
7
8   @Get()
9   getHello(): string {
10     return this.appService.getHello();
11   }
12 }
```

The class AppController has a method getHello The class AppController has a constructor that initialize appService The method getHello uses the class atribute appService via this

## ▒▒▒▒▒▒▒ Code exploration

```
1 import { Controller, Get } from '
  @nestjs/common';
2 import { AppService } from './app.service';
3
4 @Controller()
5 export class AppController {
6   constructor(private readonly appService:
  AppService) {}
7
8   @Get()
9   getHello(): string {
10     return this.appService.getHello();
11   }
12 }
```

1. What is appService an instance or class?

Instance (Line 6 appService: AppService)

## ▒▒▒▒▒▒▒ Code exploration

```
1 import { Controller, Get } from '
  @nestjs/common';
2 import { AppService } from './app.service';
3
4 @Controller()
5 export class AppController {
6   constructor(private readonly appService:
  AppService) {}
7
8   @Get()
9   getHello(): string {
10     return this.appService.getHello();
11   }
12 }
```

2. Which class does it derive from?

From AppService (Line 6 appService: AppService)

## ▒▒▒▒▒▒▒ Code exploration

```
1 import { Controller, Get } from '
  @nestjs/common';
2 import { AppService } from './app.service';
3
4 @Controller()
5 export class AppController {
6   constructor(private readonly appService:
  AppService) {}
7
8   @Get()
9   getHello(): string {
10     return this.appService.getHello();
11   }
12 }
```

3. Which method does AppService has?

getHello (Line 10 appService.getHello())

## ▒▒▒▒▒▒▒ Code exploration

```
1 import { Controller, Get } from '
  @nestjs/common';
2 import { AppService } from './app.service';
3
4 @Controller()
5 export class AppController {
6   constructor(private readonly appService:
  AppService) {}
7
8   @Get()
9   getHello(): string {
10     return this.appService.getHello();
11   }
12 }
```

4. How do we know appService.getHello is a method and not an attribute?

Parenthesis means a function invocation

## ▒▒▒▒▒▒▒ Code exploration

```
1 import { Controller, Get } from '
  @nestjs/common';
2 import { AppService } from './app.service';
3
4 @Controller()
5 export class AppController {
6   constructor(private readonly appService:
  AppService) {}
7
8   @Get()
9   getHello(): string {
10     return this.appService.getHello();
11   }
12 }
```

5. Which data type does appService.getHello() returns?

String, getHello result is directly returned by AppController.getHello (Line 9 `getHello(): string`)

## Code exploration

```
1 import { Controller, Get } from '
  @nestjs/common';
2 import { AppService } from './app.service';
3
4 @Controller()
5 export class AppController {
6   constructor(private readonly appService:
  AppService) {}
7
8   @Get()
9   getHello(): string {
10     return this.appService.getHello();
11   }
12 }
```

6. Finally even if we don't understand the notation what does Line 8 do?

It linkes the method getHello (Line 9) to the request method Get

## ▒▒▒▒▒▒▒ Code exploration

```
1 import { Controller, Get } from '
  @nestjs/common';
2 import { AppService } from './app.service';
3
4 @Controller()
5 export class AppController {
6   constructor(private readonly appService:
  AppService) {}
7
8   @Get()
9   getHello(): string {
10     return this.appService.getHello();
11   }
12 }
```

@<Name> are called Decorators it allows to expand the properties of classes, methods, even parameters

7. Which type of decorator is @Get?

It's a method decorator, affecting the behaviour of getHello()

```
1 import { Controller, Get } from '
  @nestjs/common';
2 import { AppService } from './app.service';
3
4 @Controller()
5 export class AppController {
6   constructor(private readonly appService:
  AppService) {}
7
8   @Get()
9   getHello(): string {
10     return this.appService.getHello();
11   }
12 }
```

8. Given that we know that getHello method handles our request and the responses we get on postman, what must be the result of this.appService.getHello();

Hello World! which end up being the response body

```
1 import { Controller, Get } from '
  @nestjs/common';
2 import { AppService } from './app.service';
3
4 @Controller()
5 export class AppController {
6   constructor(private readonly appService:
  AppService) {}
7
8   @Get()
9   getHello(): string {
10     return this.appService.getHello();
11   }
12 }
```

9. Where is located the definition of AppService?

src/app.service.ts

## ▒▒▒▒▒▒▒ Code exploration

A basic controller with a single route.

```
1 import { Controller, Get } from '
  @nestjs/common';
2 import { AppService } from './app.service';
3
4 @Controller()
5 export class AppController {
6   constructor(private readonly appService:
  AppService) {}
7
8   @Get()
9   getHello(): string {
10     return this.appService.getHello();
11   }
12 }
```

Let's try to draft our own version of the service!

## ▒▒▒▒▒▒▒ Code exploration

src/app.service.ts

> A basic service with a single method.

```
import { Injectable } from '@nestjs/common';

@Injectable()
export class AppService {
  getHello(): string {
    return 'Hello World!';
  }
}
```

## ▒▒▒▒▒▒▒ Code exploration

src/app.service.ts

> A basic service with a single method.

```ts
import { Injectable } from '@nestjs/common';

@Injectable()
export class AppService {
  getHello(): string {
    return 'Hello World!';
  }
}
```

What does export do?

export allow to use the class in other files

## ▒▒▒▒▒▒▒ Code exploration

`src/app.service.ts`

> A basic service with a single method.

```
1 import { Injectable } from '@nestjs/common';
2
3 @Injectable()
4 export class AppService {
5   getHello(): string {
6     return 'Hello World!';
7   }
8 }
```

What does `getHello(): string {...}` mean?

`getHello` is the method name
`()` doesnt receive parameters
`: string` the method returns a string

## ▒▒▒▒▒▒▒ Code exploration

- Controllers (app.controller.ts) == Access Layer
  - Receive and handle client requests, defining http methods, and resource path's

- Services (app.service.ts) == Process/Service Layer
  - Define business logic by orquestrating reutilizable processes

# Homework

Modify the existing endpoint to return "Hola Mundo" instead of "Hello World!"